

Sprint 3 - Complete Task & Workflow Overview

Team 1: Trade Management & Analytics

User Story 5: Staff Confirm Drop-off & Collection

*As a **staff member**, I want to confirm both item drop-off and collection so that transactions are properly tracked and completed.*

User Flow

1. Buyer reserves item → creates "pending" transaction
2. Buyer and seller message to agree on meeting time
3. One of them books a trade slot → links to transaction → status becomes "in_progress"
4. Both parties arrive at facility at booked time
5. Staff verifies exchange happened → clicks "Complete Transaction"
6. Transaction status → "completed" → enables rating for both parties

New Table: transactions

Column	Type	Why
id	uuid	Unique identifier for each transaction
buyer_id	uuid	Links to the student buying
seller_id	uuid	Links to the student selling
listing_id	uuid	Links to the item being traded
type	text	sale / trade / either - matches listing's sale_type

Column	Type	Why
status	text	Controls flow: pending → in_progress → completed / cancelled
slot_id	uuid	Links to the booked trade slot (optional)
dropoff_confirmed	boolean	Staff confirms item was handed over
collection_confirmed	boolean	Staff confirms item was received
created_at	timestamp	Track when transaction started

Transaction Status

Status	Meaning
pending	Buyer initiated reserve/buy
in_progress	Slot booked, awaiting exchange
completed	Staff confirmed both drop-off and collection
cancelled	Transaction aborted

Transaction Types

Listing sale_type	Button Shown	Creates Transaction Type
"Sale"	"Buy Now"	"sale"
"Trade"	"Reserve"	"trade"
"Either"	Both buttons	matches buyer choice

Note: For Sale items, "Buy Now" currently works the same as "Reserve" (payment deferred to later). Both buttons create a "pending" transaction.

Files to Create

Backend Developer:

1. Create models/transaction.js - Database operations
 - createTransaction({ buyer_id, seller_id, listing_id, type })
 - getTransactionById(id)
 - getTransactionsByUser(user_id)
 - updateTransactionStatus(id, status)
 - confirmComplete(id)
2. Create routes/transactions.js - API endpoints

Method	Endpoint	What happens
POST	/transactions	Buyer clicks "Reserve" - creates transaction
GET	/transactions	User views their transactions (as buyer or seller)
GET	/transactions/:id	Get details of one transaction
PUT	/transactions/:id/status	Update status manually (e.g., cancel)
PUT	/transactions/:id/complete	Staff confirms both drop-off AND collection

Frontend Developer:

1. Create transaction display - staff can view "in_progress" transactions
2. Add "Complete Transaction" button on transaction details
3. Show transaction status badge on UI

Files to Modify

- routes/slots.js - Add ability to link a slot booking to a transaction

Open Questions for Team

- Who can cancel a transaction? (buyer only, seller only, or either party?)
- What if only one party shows up? Should staff be able to mark partial completion?

User Story 4: Admin Analytics

*As an **admin**, I want to view completed transactions over time and identify popular item categories so that I can track growth and understand trends.*

User Flow

1. Admin logs in
2. Navigates to analytics dashboard
3. Views: total completed transactions, trends over time, top categories
4. Can filter by time period (week/month/year)
5. Can export data as CSV

Technical Requirements

No new tables needed - queries the transactions table joined with listings.

Files to Create

Backend Developer:

1. Create routes/analytics.js - API endpoints

Method	Endpoint	Returns
GET	/analytics/transactions	Completed transactions grouped by day/week/month
GET	/analytics/categories	Count of transactions grouped by listing category
GET	/analytics/summary	Total completed, total value, average per day

2. Ensure only Admin role can access these endpoints

Frontend Developer:

3. Create pages/admin-dashboard.html - Frontend dashboard page
4. Create scripts/analytics.js - Frontend logic for charts/data
5. Display: total transactions, trends chart, popular categories
6. Add CSV export button

Open Questions for Team

- What specific time periods should be available for filtering?
- Which metrics are most important for the trade facility utilisation report?

Team 1 Workflow

Task	Who
1. Create models/transaction.js	Backend
2. Create routes/transactions.js	Backend
3. Modify routes/slots.js - link to transaction	Backend
4. Create routes/analytics.js	Backend
5. Ensure Admin role checks on analytics	Backend
6. Create pages/admin-dashboard.html	Frontend
7. Create scripts/analytics.js	Frontend
8. Add CSV export functionality	Frontend
9. Integrate and test together	Both

Team 2: Discovery & Listing Management

User Story 2: Purchase / Reserve Item

*As a **student buyer**, I want to purchase or reserve an item so that I can initiate a transaction with the seller.*

User Flow

1. Buyer browses listings on dashboard or search
2. Clicks on an item they're interested in
3. Clicks "Reserve" or "Buy Now"
4. System validates: item is available, buyer isn't the seller

5. Transaction created with status "pending"
6. Listing status changes to "reserved"
7. Buyer can now message seller to arrange details
8. Both agree on time, book trade slot → transaction becomes "in_progress"

Update Table: listings

Column	Type	Why
status	text	Tracks if item is available/reserved/unavailable

Listing Status

Status	When
available	Default when listing first created
reserved	When any buyer initiates a transaction
unavailable	When transaction completes

Note: When a transaction is *cancelled*, the listing status automatically reverts to *available*.

Files to Modify

Backend Developer:

- Modify models/listing.js - Add function to update status
 - updateListingStatus(id, status)
- Modify routes/listings.js - Add reserve endpoint

Method	Endpoint	Returns
POST	/listings/:id/reserve	Creates transaction, updates listing to reserved

- Validate:
 - Only if listing.status = 'available'
 - Current user cannot reserve their own listing
 - If transaction already exists for that listing (status not completed/cancelled), block new reservation

Frontend Developer:

- Modify components/card.js - Dynamic buttons

- If sale_type = "Sale" → show "Buy Now"
 - If sale_type = "Trade" → show "Reserve"
 - If sale_type = "Either" → show both options
- Handle button click:
 - Call reserve endpoint
 - Show success/error feedback
 - Refresh listing display

Button Mapping

Listing sale_type	Button Shown	Creates Transaction Type
"Sale"	"Buy Now"	"sale"
"Trade"	"Reserve"	"trade"
"Either"	Both buttons	matches choice

Open Questions for Team

- Should there be a timeout? (e.g., reservation expires after 48 hours if no activity)
- Can the seller cancel the reservation?

User Story 3: Delete Listings

As a **student**, I want to delete my listings so that I can remove unavailable items.

User Flow

1. Seller goes to "My Listings" page
2. Sees all their active listings
3. Clicks "Delete" on an item
4. System checks: does this listing have an active transaction?
 - a. If YES → Show error: "Cannot delete - transaction in progress"
 - b. If NO → Allow deletion

Technical Requirements

Backend Developer:

- Ensure DELETE endpoint checks for active transactions:
 1. Fetch listing by ID

2. Query: any transaction where listing_id = X AND status NOT IN ('completed', 'cancelled')?
3. If yes → return 400 error: "Cannot delete listing with active transaction"
4. If no → proceed with delete

Files to Create

1. pages/my-listings.html - New page showing user's listings with delete buttons
 - Display only current user's listings
 - Show delete button per listing
 - Show "Cannot delete" message if transaction active

Files to Modify

- dashboard.html sidebar - "My Listings" link from `#` to `my-listings.html`

Open Questions for Team

- Should deletion be soft-delete (status = unavailable) or hard-delete from database?

Team 2 Workflow

Task	Who
1. Add updateListingStatus to models/listing.js	Backend
2. Add reserve endpoint to routes/listings.js	Backend
3. Add validation: available, not own listing	Backend
4. Add transaction check to DELETE endpoint	Backend
5. Create pages/my-listings.html	Frontend
6. Update card buttons in components/card.js	Frontend
7. Handle button click + feedback	Frontend
8. Fix sidebar "My Listings" link	Frontend
9. Integrate and test together	Both

Team 3: Transactions & Communication

User Story 6: In-App Messaging

*As a **user (buyer or seller)**, I want to communicate through messaging before booking or completing a trade so that we can clarify and finalize transaction details.*

User Flow

1. Buyer reserves item → creates transaction
2. Buyer or seller opens "Messages"
3. They see conversation with this user
4. They can send messages about the transaction
5. Messages persist and are visible only to both parties

Update Table: messages

Column	Type	Why
transaction_id	uuid (nullable)	Links message to specific transaction for context

Files to Modify

Backend Developer:

- Update models/message.js - Add transaction_id field
- Modify routes/messages.js - Allow passing transaction_id when sending
 - POST /messages/send - Accepts optional transaction_id
 - Validate: only participants can send messages linked to that transaction

Frontend Developer:

- Show transaction context in messaging (optional: "Chatting about: [item title]")
- Ensure existing message UI works with updated API

Open Questions for Team

- Should messages be real-time (WebSocket) or refresh-based?
- Do you want to show "transaction context" in the message thread header?

User Story 1: Ratings and Trust System

*As a **user**, I want to rate and leave a review for another user after a transaction so that I can share my experience and help build trust within the platform.*

User Flow

1. Transaction marked "completed" by staff
2. Both buyer and seller can now rate each other
3. Each sees option to rate the other (stars + review text)
4. Submit rating
5. Rating stored, user's average updated
6. Future buyers can see seller's average rating on listings

New Table: ratings

Column	Type	Why
id	uuid	Unique identifier
rater_id	uuid	User leaving the rating
rated_user_id	uuid	User being rated
transaction_id	uuid	Which transaction this rating is for
rating	integer	1-5 stars
review	text	Written feedback
created_at	timestamp	When rating was submitted

Files to Create

Backend Developer:

1. Create models/rating.js - Database operations
 - createRating({ rater_id, rated_user_id, transaction_id, rating, review })
 - getRatingsForUser(user_id) - Returns ratings + calculated average
2. Create routes/ratings.js - API endpoints

Method	Endpoint	Returns
POST	/ratings	Submit rating after completed transaction
GET	/ratings/:userId	Get all ratings for a user + average

3. Validation rules:

- Transaction must exist AND status = 'completed'
- User must be either buyer OR seller in that transaction
- One rating per transaction per user (no duplicates)
- Rating value must be 1-5

Frontend Developer:

1. Modify components/card.js - Show seller rating on listing cards
 - Display: "★ 4.5 (12 reviews)" next to seller name
2. Create rating UI (after transaction completes)
 - Show "Rate [user]" button to both parties
 - Star selector (1-5) + text area
 - Submit button

Open Questions for Team

- Should admins be able to delete inappropriate reviews?
- Can users edit or delete their own reviews?

Team 3 Workflow

Task	Who
1. Add transaction_id to models/message.js	Backend
2. Update /messages/send to accept transaction_id	Backend
3. Create models/rating.js	Backend
4. Create routes/ratings.js	Backend
5. Add validation: completed transaction, participant check	Backend

Task	Who
6. Show ratings on components/card.js	Frontend
7. Ensure messaging UI works	Frontend
8. Create rating submission UI	Frontend
9. Integrate and test together	Both

File Summary - All Teams

Team	New Files	Modify Files
Team 1	models/transaction.js, routes/transactions.js, routes/analytics.js, pages/admin-dashboard.html, scripts/analytics.js	routes/slots.js
Team 2	pages/my-listings.html	models/listing.js, routes/listings.js, components/card.js, dashboard.html
Team 3	models/rating.js, routes/ratings.js	routes/messages.js, models/message.js, components/card.js

Recommended Team Workflow Pattern

Step	Task
1	Backend creates model (database operations)
2	Backend creates API routes
3	Frontend builds/modifies UI
4	Both integrate and test together

Work on **one task at a time**. The Backend developer goes first to create the foundation, then Frontend builds the UI that uses those APIs, then they meet in the middle to integrate and verify the full flow works.